

Project 1 — The Grader

Implementation Guide & Git Workflow — Code Review Comments

CS496 — AI Engineering | Dr Abdeldjalil Labed | MedTech | Fall 2026

Team: Skander · Khalil · Yasmine · Abdallah

This guide picks up where the team plan left off: the task is locked in as **Code Review Comment Quality**. Everything below is written so each of the four of you can open your own branch today, work independently for a stretch, and merge into main without stepping on each other's files.

The system in one line: given a code snippet, the system writes one review comment. **The judge in one line:** it grades that comment as correct (catches a real issue, or correctly says there isn't one) and useful (specific, actionable, well-toned), returning fixed-schema JSON.

1. Target Repository Structure

This is the shape the repo should have by the end of Week 4 — set it up on Day 1 so nobody's PR fights over folder layout later.

```
the-grader/
|-- README.md                Abdallah - setup, run, results, contributions
|-- data/
|   |-- golden_set.jsonl     Skander - 150+ labelled items
|   |-- labelling_guide.md   Skander - the 2-page guide
|   |-- agreement_report.md  Skander - kappa + dropped items
|   |-- build_dataset.py     Skander - sourcing / sampling script
|-- harness/
|   |-- schema.py            Khalil - shared structured-output schema
|   |-- system.py            Khalil (+Abdallah's prompts) - generates comments
|   |-- judge_client.py      Khalil (+Yasmine's prompts) - calls the judge
|   |-- run.py               Khalil - the one-command end-to-end runner
|   |-- reliability.py       Yasmine - agreement + bias-test scripts
|-- prompts/
|   |-- system_prompt_v1.md ... Abdallah - versioned system prompts
|   |-- judge_prompt_v1.md ... Yasmine - versioned judge prompts
|   |-- CHANGELOG.md         Abdallah - every change <-> score delta
|-- results/
|   |-- <run_id>.jsonl        written by harness/run.py, never edited by hand
|-- report.pdf               Abdallah assembles, all four review
|-- postmortem.md            all four contribute
```

2. Git & GitHub Workflow

2.1 One-time setup (do this together, first session)

- One person creates the repo on GitHub, adds the other three as collaborators (or make it a GitHub org/team so everyone has equal standing).
- Protect main: Settings → Branches → Branch protection rule on main → require a pull request before merging, require 1 approval.
- Agree on the schema in `harness/schema.py` together *before* anyone writes code against it — this is the one contract every track depends on.
- Add a shared `.gitignore` (Python + node + `.env`) so nobody accidentally commits an API key.

```
git init
git branch -M main
git remote add origin git@github.com:<org>/the-grader.git
git add README.md .gitignore
git commit -m "chore: initial scaffold"
git push -u origin main
```

2.2 Branch naming — one branch per track

Student	Branch	Touches
Skander	feat/golden-dataset	data/
Khalil	feat/harness-core	harness/schema.py, system.py, run.py
Yasmine	feat/judge-reliability	harness/judge_client.py, reliability.py, prompts/judge_prompt_*.md
Abdallah	feat/prompt-iteration	prompts/system_prompt_*.md, CHANGELOG.md, README.md, cost model

Rule of thumb so branches don't collide: each file has exactly one owner branch. If you need to touch someone else's file, ping them or open a tiny separate PR against it — don't edit it from your own branch.

2.3 Commit message convention

Use conventional commits so the log itself becomes a readable history of the project (and doubles as evidence for the README's Contributions section):

```
feat(harness): add structured-output schema for system + judge
data(golden-set): add first 60 labelled snippets
docs(prompts): log v2 system prompt, dev score 0.71 -> 0.79
fix(judge): correct schema mismatch on empty comment case
chore: freeze repo for Week 5 demo
```

2.4 Pull requests

- Open a PR as soon as a track hits a working milestone — don't wait until the whole track is "done".
- One teammate (not the author) reviews and approves before merge — with 4 people this costs almost nothing and is exactly the kind of process the grading rubric rewards under "Documented".
- PR description template: what changed, how you tested it, which score it affects (if any).
- Squash-merge into main so main's history stays one clean commit per feature.

2.5 Dependency order — who merges first

The four tracks aren't fully independent: the harness needs the schema before anyone can test against it, and the judge needs the harness's calling convention. Merge in this order to avoid rebasing headaches:

Order	Branch merged	Why it has to go first
1	feat/harness-core (schema only, small PR)	Everyone else's code imports this schema.
2	feat/golden-dataset (in progress, parallel)	No hard dependency — starts Day 1 in parallel.
3	feat/prompt-iteration (v1 baseline)	Harness needs a real system to call for its first end-to-end test.
4	feat/harness-core (full runner)	Now runnable end-to-end against v1 prompts + dev-set snippets.
5	feat/judge-reliability	Plugs into the now-working harness to score real outputs.

Weekly integration point: every Friday, all four `git pull origin main`, run the full pipeline together, and fix any integration break immediately — never let a branch drift more than a few days from main.

3. Skander's Track — Golden Dataset

Skander — Golden Dataset (branch: feat/golden-dataset)

Step by step

- 1 Collect 150–180 code snippets (Python + one more language) from past assignments and small public repos — aim for a mix of buggy and clean code, roughly 50/50.
- 2 For each snippet, write one candidate review comment. Deliberately vary quality: some genuinely good, some wrong, some vague/padded — you need both classes well represented.
- 3 Draft `labelling_guide.md` (max 2 pages): definitions of *correct* and *useful*, plus 4–6 worked examples (good, bad, borderline with reasoning).
- 4 Label every item yourself first (`label_1`), then have a teammate independently label a full pass (`label_2`) without seeing your labels.
- 5 Compute agreement and write `agreement_report.md`.
- 6 Lock a 70/30 dev/test split and tag each row with `"split": "dev" or "test"`.

Agreement calculation

```
from sklearn.metrics import cohen_kappa_score
```

```
kappa = cohen_kappa_score(labeler_1_labels, labeler_2_labels)
pct_agree = sum(a == b for a, b in zip(labeler_1_labels, labeler_2_labels)) / len(labeler_1_labels)
```

Commit sequence (example)

```
git checkout -b feat/golden-dataset
git commit -m "data(golden-set): add build_dataset.py sourcing script"
git commit -m "data(golden-set): add first 80 snippets + candidate comments"
git commit -m "data(golden-set): add labelling_guide.md v1"
git commit -m "data(golden-set): complete double-labelling, add agreement_report.md"
git commit -m "data(golden-set): finalize dev/test split at 160 items"
git push -u origin feat/golden-dataset # open PR, tag a reviewer
```

Definition of done

- 150+ items in `data/golden_set.jsonl`, every row double-labelled.
- Agreement report has both % agreement and kappa.
- `labelling_guide.md` is under 2 pages and has worked examples.

4. Khalil's Track — Harness Core

Khalil — Harness Core (branch: feat/harness-core)

Step by step

- 1 Lock the shared schema in `harness/schema.py` and push it first, alone, as the smallest possible PR — everyone else blocks on this.
- 2 Write `system.py`: one function that takes a code snippet and returns a structured comment (calls whichever prompt Abdallah has pushed to prompts/).
- 3 Write `judge_client.py`: one function that takes (code, comment) and returns the fixed-schema verdict (calls Yasmine's judge prompt once it exists — stub it with a fake response until then).
- 4 Write `run.py`: loads the golden set, filters by split, calls `system` → `judge` for every item, writes one row per item to results/, and prints an aggregate summary.
- 5 Add cost and latency logging to every call (wall-clock time + token counts from the API response).
- 6 Verify the whole thing runs from a clean clone with exactly one command.

Shared schema

```
SYSTEM_OUTPUT_SCHEMA = {
    "comment": "string"
}

JUDGE_OUTPUT_SCHEMA = {
    "verdict": "good" | "bad",
    "correct": true | false,
    "useful": true | false,
    "reason": "string, <= 30 words"
}
```

Runner skeleton

```
def run_harness(golden_path, split, out_path):
    items = [i for i in load_jsonl(golden_path) if i["split"] == split]
    rows = []
    for item in items:
        t0 = time.time()
        sys_out = call_system(item["code_snippet"])
        judge_out = call_judge(item["code_snippet"], sys_out["comment"])
        rows.append({
            "id": item["id"],
            "system_output": sys_out,
            "judge_verdict": judge_out,
            "latency_s": time.time() - t0,
            "cost_usd": estimate_cost(sys_out, judge_out),
        })
    write_jsonl(out_path, rows)
    return summarize(rows) # pass rate, per-language breakdown, mean cost/latency
```

Commit sequence (example)

```
git checkout -b feat/harness-core
git commit -m "feat(harness): add structured-output schema"
git commit -m "feat(harness): add system.py caller"
git commit -m "feat(harness): add judge_client.py with stub judge"
git commit -m "feat(harness): add end-to-end run.py with jsonl output"
git commit -m "feat(harness): add cost + latency logging"
git push -u origin feat/harness-core # PR #1 for schema lands first, rest follow
```

Definition of done

- `python harness/run.py --split` test runs clean from a fresh clone with no manual steps.
- One JSONL row per item, never a single aggregate-only output.
- Cost and latency are logged for every call, not estimated after the fact.

5. Yasmine's Track — LLM Judge & Reliability

Yasmine — Judge & Reliability (branch: feat/judge-reliability)

Step by step

- 1 Write the pointwise judge prompt against the labelling guide (Skander's guide is the source of truth for what "correct"/"useful" mean).
- 2 Run the pointwise judge on every double-labelled item in the dev set; compute agreement with the human labels.
- 3 Build a second, *compare-mode* judge prompt for the bias tests specifically (see below) — it doesn't ship in the main harness, it's a diagnostic tool.
- 4 Run the position-bias test and the verbosity-bias test (procedures below); write up the numbers and the honest failure-mode paragraph.

Pointwise judge prompt (used by the harness)

You are grading a code review comment against a fixed guide.
A comment is **CORRECT** if it identifies a real issue present in the code, or correctly states there is none. A comment is **USEFUL** if it is specific, actionable, and professionally toned.

Code:
{code}

Comment to grade:
{comment}

Return only JSON:
{ "verdict": "good"|"bad", "correct": true|false, "useful": true|false, "reason": "<=30 words" }

Compare-mode judge prompt (bias-testing only)

Given the code below and two candidate review comments, say which is better.

Code:
{code}
Comment A: {comment_a}
Comment B: {comment_b}

Return only JSON:
{ "winner": "A"|"B"|"tie", "reason": "<=30 words" }

Position-bias test procedure

- 1 From the golden set, take N pairs: one "good"-labelled comment and one "bad"-labelled comment on the same snippet.
- 2 Ask compare-mode with (A=good, B=bad); record the winner.
- 3 Ask again with the order flipped (A=bad, B=good); record the winner.
- 4 Flag a flip whenever the winner tracks position instead of quality across the two runs.
- 5 Report: % of pairs where the verdict flipped purely from the swap.

Verbosity-bias test procedure

- 1 Take comments labelled "bad" in the golden set.
- 2 Pad each with 2–3 generic sentences that add zero new technical information.
- 3 Re-run the padded version through the pointwise judge.
- 4 Report: % of padded items whose verdict flips to "good" (or whose "useful" flips to true) despite no real improvement.

Commit sequence (example)

```
git checkout -b feat/judge-reliability
git commit -m "feat(judge): add judge_prompt_v1.md (pointwise)"
git commit -m "feat(judge): wire judge_client.py to real prompt"
git commit -m "eval(judge): agreement with humans = 0.83 kappa"
git commit -m "eval(judge): add compare-mode prompt for bias tests"
git commit -m "eval(judge): position-bias report - 6% flip rate"
git commit -m "eval(judge): verbosity-bias report - 14% flip rate on padding"
git push -u origin feat/judge-reliability
```

Definition of done

- Judge-vs-human agreement reported as both % and kappa, on dev and test separately.
- Both bias numbers reported as percentages, with at least 2 concrete failing examples kept for the demo.

6. Abdallah's Track — Prompts, Cost Model & Docs

Abdallah — Prompt Iteration & Docs (branch: feat/prompt-iteration)

Step by step

- 1 Write `system_prompt_v1.md` — a simple, direct instruction to review the given code snippet and return one comment in the shared schema. Push this early so Khalil can wire the harness against a real system.
- 2 Once the harness runs end-to-end on the dev set, read the failure cases (false positives, missed bugs, vague comments) and write v2.
- 3 For every version, add one line to `CHANGELOG.md`: what changed, and the dev-set score before/after.
- 4 Build the cost model script and note.
- 5 Once `v_final` is frozen, run the one clean, single test-set evaluation (never touch the test set before this point).
- 6 Assemble `report.pdf` (max 4 pages), `postmortem.md`, and the README's setup/run/results/Contributions sections; do the final clean-clone check before the repo is frozen.

CHANGELOG.md format

```
## v2 - 2026-XX-XX
Changed: explicitly told the model to say "no issue found" on clean code
         instead of inventing one.
Dev score: 0.71 -> 0.79 (correct: 0.68 -> 0.83, useful: 0.75 -> 0.76)

## v1 - 2026-XX-XX
Baseline prompt: "Review this code and point out any issues."
```

Cost model

For each run, from the harness's logged token counts:

```
cost_per_item = (input_tokens * price_in + output_tokens * price_out) / 1_000_000
cost_per_1k   = cost_per_item * 1000
cost_at_100x  = cost_per_1k * 100 * <expected_daily_volume_multiplier>
```

Note in the report that unit cost does not always scale linearly at 100× — mention batching, prompt caching, or rate limits as the realistic offsets. That one sentence is usually the difference between a cost section that looks copy-pasted and one that looks reasoned through.

Commit sequence (example)

```
git checkout -b feat/prompt-iteration
git commit -m "feat(prompts): add system_prompt_v1.md baseline"
git commit -m "docs(prompts): v2 - fix false-positive rate on clean code, +8% dev score"
git commit -m "docs(prompts): v3 - tighten tone guidance, +3% useful score"
git commit -m "docs: add cost_model.py and cost section notes"
git commit -m "docs: assemble report.pdf and postmortem.md"
git commit -m "chore: final clean-clone verification before freeze"
git push -u origin feat/prompt-iteration
```

Definition of done

- Every prompt version has a CHANGELOG entry with a before/after dev-set score — no version without a number.
- The test set was scored exactly once, after the prompt was frozen.
- README lets a stranger clone the repo and reproduce your headline result with one setup command and one run command.

7. Final Integration Checklist — Before You Freeze the Repo

- All four branches merged into main, in the dependency order above, each via a reviewed PR.
- `git clone` into a brand-new folder and run the harness with zero manual fixes.
- results/ contains at least one full test-set run, timestamped, never overwritten.
- prompts/CHANGELOG.md tells the whole prompt story end-to-end, in order.
- No API keys anywhere in the git history (not just the latest commit — check with a quick `git log -p | grep -i key`).
- README's Contributions section lists what each of Skander, Khalil, Yasmine and Abdallah actually built — the commit history above should make this trivial to write honestly.
- postmortem.md has real answers, including at least one thing that didn't work.

Reminder for demo day: rehearse the position- and verbosity-bias examples yourselves before the instructor tries the same tricks live.